

آلة الكرات

ابتكر "عمر" آلة كرات لتسلية المشاركين في مسابقة IOI. فيما يلي الهيكل الداخلي للآلة:

- تتكون الآلة N عقدة (**nodes**)، مرقمة من 0 إلى $N - 1$. وتُسمى العقدة $N - 1$ جذر (**root**).
 - بالنسبة لكل عقدة u بحيث $0 \leq u < N - 1$ ، فإن الوالد للعقدة u هو العقدة $P[u]$ ذات $P[u] > u$ ، والعقدة u هي فرع لـ $P[u]$. الجذر ليس له أصل. وتسمى العقدة التي ليس لها فروع ورقة.
- أنت لا تعرف N أو الوالد $P[u]$ لأي عقدة u . بدلاً من ذلك، يخبرك عمر بـ M ، وعدد الأوراق، وأن مؤشرات الأوراق هي $0, 1, \dots, M - 1$. مهمتك هي تحديد البنية الداخلية للآلة من خلال تشغيلها.
- يمكن لكل عقدة في الآلة أن تحتوي على كرة واحدة على الأكثر، ولكل كرة قيمة وهي عدد صحيح غير سالب. نقول إن العقدة لها القيمة x إذا كانت تحتوي على كرة بقيمة x . تكون العقدة مشغولة إذا كانت تحتوي على كرة؛ وإلا فهي خالية. في البداية، تكون كل عقدة خالية.
- يمكنك تنفيذ نوعين من العمليات:

- **insert(U, X)**: محاولة إدخال كرة جديدة بقيمة X عند الورقة U .
 - إذا كانت الورقة U مشغولة، فإن العملية تُرجع false دون إدراج الكرة.
 - إذا كانت الورقة U خالية، تُوضع الكرة في العقدة U . ثم تنتقل الكرة بشكل متكرر إلى الوالد لعقدتها الحالية طالما أن عقدة الوالد تلك خالية. تتوقف الحركة عندما تصل الكرة إلى الجذر أو إلى عقدة تكون عقدة الوالد الخاصة مشغولة. تُرجع العملية true.
- **collect()**: إذا كان الجذر خالياً (مما يعني أن الآلة فارغة)، تُرجع collect مصفوفة فارغة. وإلا، فإن الإجراء التكراري traverse الموصوف أدناه يجمع قيم جميع الكرات الموجودة حالياً في الآلة في مصفوفة S . في البداية، تكون المصفوفة S فارغة ويتم استدعاء $traverse(N - 1)$.

traverse(u):

أضف قيمة الكرة الموجودة في العقدة u إلى نهاية S

لنفترض أن $c[u]$ هي قائمة الأبناء المشغولة لـ u

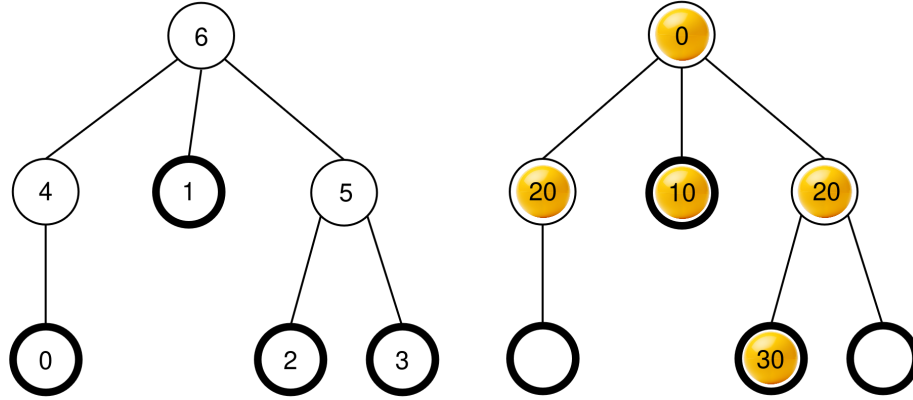
قم بترتيب $c[u]$ بترتيب غير تنازلي اعتماداً على قيم الكرات في تلك العقد (إذا كانت هناك عدة عقد لها نفس القيمة، يمكن أن تظهر بأي ترتيب)

لكل v في $c[u]$:

traverse(v)

بعد اكتمال هذا الإجراء، تتم إزالة جميع الكرات من الآلة، وتُرجع S collect.

على سبيل المثال، لنفترض وجود آلة بها $N = 7$ عقد ومصفوفة أبوية $P = [4, 6, 5, 5, 6, 6]$. تعرض الصورة على اليسار الآلة الفارغة مع مؤشرات العقد، بينما تُظهر الصورة على اليمين تكويناً محتملاً للكرات بعد سلسلة من عمليات insert (تُرد هذه السلسلة في قسم الأمثلة).



عند استدعاء $\text{collect}()$ ، تكون العقدة الجذرية (6) مشغولة، لذا يتم تهيئة S على أنها $S = []$ ويتم استدعاء $\text{traverse}(6)$.

$\text{traverse}(6)$: العقدة 6 لها القيمة 0؛ وإلحاقها بـ S يعطي $S = [0]$. العقد الفرعية للعقدة 6 هي 4 و 1 و 5، وجميعها مشغولة. قيمها هي 20 و 10 و 20، على التوالي. يؤدي الترتيب بترتيب غير تنازلي إلى الحصول على $c[6] = [1, 5, 4]$ (لاحظ أن $c[6] = [1, 4, 5]$ سيكون صحيحاً أيضاً، نظراً لأن العقدتين 4 و 5 لهما نفس القيمة). ثم تستدعي الإجراء traverse على كل عقدة في $c[6]$ بهذا الترتيب.

- **$\text{traverse}(1)$** : العقدة 1 لها القيمة 10؛ وإلحاقها بـ S يعطي $S = [0, 10]$. العقدة 1 ليس لها أبناء مشغولون، لذا ينتهي هذا الاستدعاء.

- **$\text{traverse}(5)$** : العقدة 5 لها القيمة 20؛ وإلحاقها بـ S يعطي $S = [0, 10, 20]$. العقدة 5 لها عقدة فرعية واحدة مشغولة، وهي العقدة 2، لذا فإن $c[5] = [2]$ والإجراء يستدعي $\text{traverse}(2)$.

- **$\text{traverse}(2)$** : العقدة 2 لها القيمة 30؛ وإلحاقها بـ S يعطي $S = [0, 10, 20, 30]$. العقدة 2 ليس لها أي عقدة فرعية مشغولة، لذا ينتهي هذا الاستدعاء.

- يعود التنفيذ إلى $\text{traverse}(5)$ التي قامت بمعالجة جميع العناصر التابعة في $c[5]$ ، لذا ينتهي استدعاؤها.

- **$\text{traverse}(4)$** : العقدة 4 لها القيمة 20؛ وإلحاقها بـ S يعطي $S = [0, 10, 20, 30, 20]$. العقدة 4 ليس لها عناصر تابعة مشغولة، لذا ينتهي هذا الاستدعاء.

اكتملت جميع الاستدعاءات ولم يعد هناك أي عقدة أخرى لتتم معالجتها. وأخيراً، تتم إزالة كل كرة من الآلة وتُرجع الدالة collect المصفوفة $S = [0, 10, 20, 30, 20]$.

مهمتك هي تحديد عدد العقد N وتقديم مصفوفة من عقد الوالد $R = [R[0], R[1], \dots, R[N-2]]$ التي نصف بنية الآلة، ولكن مع تسمية مختلفة محتملة للعقد التي ليست أوراًفاً وليست الجذر. من الناحية النظرية، يُعتبر المصفوف المبلغ عنه R صحيحاً إذا كان من الممكن تعيين تسمية مختلفة $L[u]$ مع $0 \leq L[u] < N$ لكل عقدة u في الآلة بحيث:

- $L[u] = u$ لجميع $0 \leq u < M$ و $u = N-1$ ، و
- $L[P[u]] = R[L[u]]$ لجميع $0 \leq u < N-1$.

ليس مطلوباً أن يكون $R[u] > u$. يجب أن تكون الآلة فارغة عند الإبلاغ عن الحل.

لنفترض أن K هو عدد عمليات collect التي تم إجراؤها، و B هو القيمة القصوى لأي كرة عبر جميع استدعاءات insert. ولنفترض أن $C = K + B$. إذن يجب ألا يتجاوز C 1000. تعتمد نتيجتك في بعض المهام الفرعية على قيمة C .

تفاصيل التنفيذ

يجب عليك تنفيذ الإجراء التالي.

```
std::vector<int> find_structure(int M)
```

- M : عدد الأوراق في الآلة.
- يتم استدعاء الإجراء مرة واحدة فقط لكل حالة اختبار.
- يجب أن يُرجع الإجراء مصفوفة $R = [R[0], R[1], \dots, R[N-2]]$ بطول $N-1$ ، وهي مصفوفة صحيحة من العناصر الأبوية.

للتفاعل مع الآلة، يمكن لإجرائك استدعاء الإجرائين التاليين.

الإجراء الأول هو:

```
bool insert(int U, int X)
```

- U : مؤشر الورقة التي يجب إدراج الكرة فيها. يجب استيفاء الشرط $0 \leq U < M$.
- X : القيمة الصحيحة للكرة. يجب استيفاء الشرط $0 \leq X \leq 1000$.
- يُرجع هذا الإجراء true إذا تم إدراج الكرة بنجاح، أو false إذا كانت الورقة U مشغولة بالفعل.
- يمكن استدعاء هذا الإجراء 500 000 مرة كحد أقصى.

الإجراء الثاني هو:

```
std::vector<int> collect()
```

- يجمع القيم الموجودة على جميع الكرات التي تم إدخالها، ويفرغ الآلة، ويعيد المصفوفة التي تم جمعها S .

إذا تجاوزت قيمة C في أي لحظة أثناء تنفيذ برنامجك 1000، فسيحصل حلك على الحكم

Output isn't correct: Too many resources used

يتم تحديد بنية الآلة قبل استدعاء find_structure. يُعتبر المُصنَّف حتمياً بمعنى أنه إذا قمت بتشغيله مرتين، وأدى كلا التشغيلين نفس تسلسل العمليات، فإن استدعاءات collect ستُرجع نفس المصفوفات.

القيود

- $2 \leq N \leq 1000$

$$1 \leq M \leq 200$$

$$M < N$$

المهام الفرعية

المهمة الفرعية	النتيجة	قيود إضافية
1	5	لدى الجذر M فرع بالضبط
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	لا توجد قيود إضافية

في المهمتين الفرعيتين 3 و 4، تعتمد نتيجتك على قيمة C على النحو التالي.

المهمة الفرعية 3 (25 نقطة)

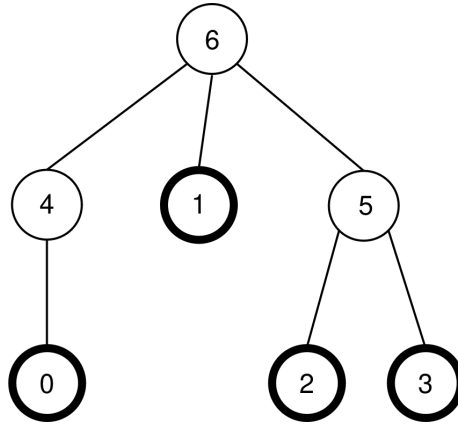
الحدود	النتيجة
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

المهمة الفرعية 4 (60 نقطة)

الحدود	النتيجة
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

مثال

لنفترض أن الجهاز هو نفسه المذكور في وصف المهمة، أي $N = 7$ و $M = 4$ و $P = [4, 6, 5, 5, 6, 6]$. ويظهر الجهاز مرة أخرى في الشكل التالي:



يقوم برنامج التصحيح بإجراء الاستدعاء التالي:

```
find_structure(4)
```

ويمكن للإجراء إجراء سلسلة الاستدعاءات التالية:

1. **insert(0, 0)**: الورقة 0 خالية، لذا يتم وضع كرة بقيمة 0 في الورقة 0. العقدة $P[0] = 4$ خالية، لذا تنتقل الكرة إلى العقدة 4.
- العقدة $P[4] = 6$ خالية، لذا تنتقل الكرة إلى العقدة 6. وبما أن العقدة 6 هي الجذر، يتوقف الانتقال. يعود الاستدعاء بقيمة **true**.
2. **insert(3, 20)**: الورقة 3 خالية، لذا يتم وضع كرة بقيمة 20 في الورقة 3. العقدة $P[3] = 5$ خالية، لذا تتحرك الكرة إلى العقدة 5. وبما أن $P[5] = 6$ مشغولة، تتوقف الحركة. ويرجع الاستدعاء بقيمة **true**.
3. **insert(1, 10)**: الورقة 1 خالية، لذا يتم وضع كرة بقيمة 10 في الورقة 1. نظرًا لأن $P[1] = 6$ مشغولة، تبقى الكرة في العقدة 1. ويرجع الاستدعاء بقيمة **true**.
4. **insert(2, 30)**: الورقة 2 خالية، لذا يتم وضع كرة بقيمة 30 في الورقة 2. نظرًا لأن $P[2] = 5$ مشغولة، تبقى الكرة في العقدة 2. ويرجع الاستدعاء بقيمة **true**.
5. **insert(1, 25)**: الورقة 1 مشغولة، لذا لا يتم وضع الكرة في الورقة 1. ويرجع الاستدعاء بقيمة **false**.
6. **insert(0, 20)**: الورقة 0 خالية، لذا يتم وضع كرة بقيمة 20 في الورقة 0. العقدة $P[0] = 4$ خالية، لذا تنتقل الكرة إلى العقدة 4. وبما أن $P[4] = 6$ مشغول، تتوقف الحركة. يقوم الإجراء بارجاع القيمة **true**.

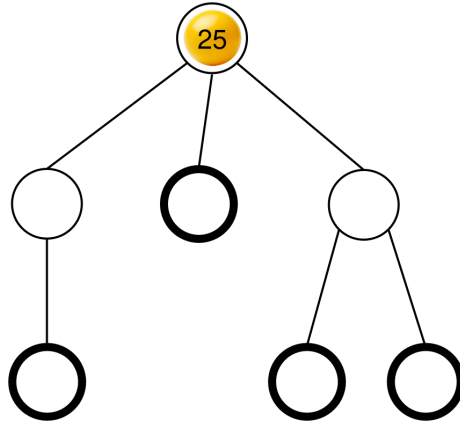
تم عرض التكوين النهائي للكرات مسبقًا في وصف المهمة.

بعد ذلك، يمكن للإجراء استدعاء:

```
collect()
```

وقد تم شرح تنفيذ هذا الاستدعاء مسبقًا في وصف المهمة، لذا فإن هذا الاستدعاء يُرجع $S = [0, 10, 20, 30, 20]$. بعد ذلك، تصبح الآلة فارغة مرة أخرى.

بعد ذلك، يمكن للإجراء استدعاء **insert(2, 25)**. الورقة 2 خالية، لذا يتم وضع كرة بقيمة 25 في الورقة 2. ثم تنتقل هذه الكرة إلى العقدة 5، ثم إلى العقدة 6، حيث تتوقف. يُرجع الاستدعاء **true**. يظهر التكوين الناتج للكرات في الشكل التالي.



أخيراً، يمكن للإجراء استدعاء `collect()` الذي يُرجع $S = [25]$ ويُفرغ الآلة.

هناك مصفوفتان أمان صحيحتان يمكن أن تُرجعهما الدالة `find_structure`: $R = P = [4, 6, 5, 5, 6, 6]$ (التي تتوافق مع التسمية $L = [0, 1, 2, 3, 5, 4, 6]$)، و $R = [5, 6, 4, 4, 6, 6]$ (التي تتوافق مع التسمية $L = [0, 1, 2, 3, 4, 5, 6]$). في هذا المثال، $K = 2$ و $B = 30$ ، لذا فإن $C = 32$.

نموذج مصحح

تنسيق الإدخال:

N M

P[0] P[1] P[2] ... P[N-2]

تنسيق الإخراج:

K B C

Q

R[0] R[1] R[2] ... R[Q-1]

هنا، Q هو طول المصفوفة R التي تُرجعها الدالة `find_structure`.